SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

AI Model Design Exercise

Code of Conduct:

I _Sk. Fawaz Ahamad / 192425390_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sk. Fawaz Ahamad

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	

Total	100%	

Signature of the Faculty Total Marks Awarded **ANSWER ANY ONE QUESTION**

S.No.	Scenario / Problem Statement	AI Model / Technique to be Developed	Expected Development Outcome
1	**Smart Loan Approval:** A bank wants to automatically determine whether a customer is eligible for a loan using income, credit score, employment status, loan amount, and repayment history.	Decision Tree	Design and implement a decision-tree-based AI model and determine the most suitable attribute-selection measure such as Information Gain, Gain Ratio, or Gini Index.
2	**Student Performance Prediction:** A university wants to predict whether a student is likely to pass or fail a course using attendance, internal marks, assignment performance, study hours, and previous academic performance.	Inductive Learning	Construct a predictive model from training examples and derive generalized decision rules from the learned patterns.
3	**Disease Risk Prediction:** A healthcare system needs to classify patients into low-risk and high-risk categories based on clinical measurements.	Statistical Learning	Design a statistical learning model, train it using patient data, and evaluate its generalization ability on unseen data.
4	**Image Recognition System:** An organization wants to automatically classify images into categories such as vehicles, animals, and buildings.	Neural Network	Design and implement a neural-network classifier by specifying the network architecture, activation functions, loss function, optimizer, and training procedure.

5	**Handwritten Character Recognition:** A system must recognize handwritten characters from scanned images.	Multilayer Neural Network	Develop a multilayer neural network and demonstrate how backpropagation updates weights to minimize classification error.
6	**Non-Linear Classification:** A dataset contains two classes that cannot be separated using a straight-line decision boundary.	SVM with Kernel Method	Design an SVM-based classifier using an appropriate kernel function and justify the selected kernel based on the characteristics of the dataset.
7	**Intelligent Recommendation Agent:** An online platform wants an agent that learns which recommendations to provide based on user responses.	Reinforcement Learning	Design an RL model by defining the states, actions, rewards, policy, and learning strategy, and demonstrate how the agent learns from user feedback.
8	**Autonomous Game Agent:** An AI agent must learn to play a game where successful actions receive positive rewards and unsuccessful actions receive penalties.	Reinforcement Learning	Develop an RL-based game agent and demonstrate how its policy and performance improve through repeated interaction with the environment.
9	**Explainable Diagnosis System:** A medical AI system predicts a disease but must also provide an explanation for its prediction.	Explanation-Based Learning	Design an explanation-based learning approach using domain knowledge and training examples to generate an understandable justification for the prediction.
10	**AI Model Selection Challenge:** A company provides a dataset for predicting customer churn.	Comparative AI Model Design	Design and compare at least two learning approaches from decision trees, statistical learning, neural networks, or kernel methods. Select the best model based on accuracy, interpretability, computational requirements, and generalization ability.

Structure of the Report:

S.No.	Report Component	What the Student Should Include
1	**Problem Statement**	Clearly describe the real-world problem, input data, expected output, and AI task to be solved.
2	**Objectives**	State the specific goals of developing the AI model and expected outcomes.
3	**Dataset Description**	Provide dataset source, number of instances, features, target variable, data types, and important attributes.
4	**Data Preprocessing**	Explain data cleaning, missing-value handling, encoding, normalization/scaling, feature selection, and train-test splitting.
5	**Selected AI Technique and Justification**	Identify the selected AI technique and justify its suitability for the problem.
6	**Model Design / Architecture**	Present the model structure, components, input/output, parameters, hyperparameters, and architecture/flow diagram.
7	**Algorithm / Pseudocode**	Provide the algorithmic steps or pseudocode showing model training and prediction/decision-making.
8	**Implementation**	Provide programming language, libraries/tools, important code sections, and implementation details.
9	**Experimental Results**	Present results using tables, graphs, sample predictions, learning curves, confusion matrices, or other relevant outputs.
10	**Performance Evaluation**	Evaluate the model using appropriate performance metrics such as accuracy, precision, recall, F1-score, MSE, RMSE, or cumulative reward.

11	**Result Analysis**	Interpret results, identify important patterns, compare actual and predicted outcomes, and discuss strengths and weaknesses.
12	**Limitations**	Discuss limitations related to dataset, model complexity, computational resources, accuracy, generalization, or deployment.
13	**Future Enhancements**	Suggest improvements such as additional data, feature engineering, hyperparameter tuning, alternative algorithms, or deployment.
14	**Conclusion**	Summarize the problem, methodology, major findings, model performance, and overall effectiveness.
15	**References**	List datasets, research papers, books, websites, documentation, libraries, and other sources used.
16	**GitHub Repository Link**	Provide the public GitHub repository URL containing the complete source code, dataset information/source, implementation, results, README, and supporting files.

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL – 2

QUESTION 1 – RUBRIC-BASED TECHNICAL ANALYSIS

L1, L2 AND L3 CACHE

Particular	Details
Course Code	CSA12
Course	Computer Architecture for Multicore Systems
Course Outcome	CO4
Question	Q1 – Compare L1, L2 and L3 cache levels
Student Name	_Sk. Fawaz_Ahamad_____
Register Number	_192425390_____
Department	Computer Science & Engineering
Date	_23/08/2026_____

Question Statement

Compare L1, L2 and L3 cache levels in terms of latency, bandwidth and throughput, and analyze their impact on processor performance.

Important Rubric Note

The supplied rubric contains 12 criteria, including several criteria normally used for AI/ML project assessment. This document retains all 12 criteria exactly in the rubric-mapping section. For this cache-architecture question, criteria that are not directly applicable to a non-AI cache-comparison problem are explicitly identified rather than inventing dataset, GitHub, or AI-model evidence. The first seven criteria are addressed with particular depth where they can be meaningfully mapped to the technical problem.

Q1. L1, L2 AND L3 CACHE – DETAILED TECHNICAL ANALYSIS

1. Problem Analysis and Domain Understanding

The performance of a multicore processor depends strongly on how quickly instructions and data can be supplied to the execution cores. Processor cores operate much faster than main memory, so directly accessing DRAM for every request would introduce substantial waiting time. A hierarchical cache system reduces this gap by keeping frequently used information closer to the processor.

L1, L2 and L3 caches form successive levels of the memory hierarchy. L1 is the smallest and fastest cache and is normally associated closely with an individual core. L2 provides greater capacity with somewhat higher latency. L3 is generally larger and is often shared among multiple cores. The central engineering problem is therefore to balance latency, capacity, bandwidth, sharing, area and power so that useful processor throughput is maximized.

- Primary objective: reduce average memory-access latency.
- Secondary objective: reduce memory stalls and unnecessary DRAM traffic.
- Performance indicators: cache hit rate, miss rate, memory-stall cycles, bandwidth utilization, IPC and application throughput.
- Decision principle: no single cache level is universally best; each level serves a different point in the hierarchy.

2. Dataset Selection and Data Understanding – Adapted to the Technical Problem

The supplied rubric names dataset selection and data understanding. This Q1 is a computer-architecture comparison rather than an AI/ML prediction problem, so no training dataset is required. To preserve the intent of the criterion, the equivalent evidence base is a structured set of cache-performance observations obtained from controlled workloads or architecture specifications.

Evidence/Input	What is Examined	Purpose
Cache specifications	Size, associativity, hierarchy level and sharing	Understand structural differences
Latency measurements	Access time at L1, L2 and L3	Quantify waiting cost
Bandwidth measurements	Data-transfer capability	Assess delivery capacity
Performance counters	Hits, misses and stall cycles	Connect cache behavior to CPU performance
Application benchmarks	Execution time and throughput	Evaluate real workload impact

For a valid experimental comparison, the workload, processor configuration, compiler conditions and measurement method should be controlled. This prevents a conclusion from being based only on theoretical cache size.

3. Data Preprocessing and Feature Engineering – Adapted

For a cache-performance study, preprocessing means preparing performance-counter and benchmark observations for fair comparison. Measurements should use consistent units and workload conditions. Derived indicators can then be calculated to expose the relationship between cache behavior and processor performance.

Raw Observation	Derived Feature / Metric	Interpretation
Cache hits and total accesses	Hit rate = hits / total accesses	Higher generally indicates better locality
Cache misses and total accesses	Miss rate = misses / total accesses	Higher values indicate more lower-level accesses
Execution cycles and useful instructions	IPC = instructions / cycles	Higher IPC indicates better execution efficiency
Memory stall cycles	Stall-cycle percentage	Shows processor waiting due to memory
Data transferred and time	Bandwidth = data / time	Shows transfer capability
Completed tasks and time	Application throughput	Shows end-to-end performance

This feature-based representation is important because cache capacity alone cannot explain processor performance. A larger cache can be useful only when it improves locality enough to offset its additional access cost, area or power.

4. AI Technique Selection and Justification – Domain-Adapted

The original rubric refers to AI technique selection. This Q1 does not require an AI model because the task is a deterministic computer-architecture comparison. Therefore, applying machine learning would add unnecessary complexity. The technically appropriate approach is comparative performance analysis supported by controlled benchmarking and processor performance counters.

Approach	Suitability for Q1	Justification
Direct analytical comparison	Excellent	Directly answers latency, bandwidth and throughput requirements.
Controlled benchmarking	Excellent	Provides workload-level performance evidence.
Performance-counter analysis	Excellent	Connects cache events to processor stalls and IPC.
Machine-learning prediction	Not required	Would not directly answer the stated comparison question.

Decision: a measurement-driven comparative method is more appropriate than an AI technique for this problem. This is a justified methodological choice because the objective is to explain and evaluate known architectural mechanisms rather than predict an unknown target.

5. Model Design / Architecture – Adapted to Cache Hierarchy

The architecture being analyzed is the processor memory hierarchy. The design can be represented as a sequence of progressively larger but slower cache levels followed by main memory.

Level	Relative Latency	Capacity	Sharing	Primary Function
L1	Lowest	Smallest	Usually core-local	Serve the hottest instructions/data
L2	Low	Larger than L1	Core or cluster dependent	Absorb L1 misses

L3	Higher than L2	Largest cache	Often shared	Reduce DRAM accesses and support shared working sets
DRAM	Much higher than cache	Large	System-level	Main working memory

Architectural flow: Processor core → L1 → L2 → L3 → DRAM. A hit at an earlier level avoids the latency of subsequent levels. Consequently, the hierarchy is designed to exploit temporal and spatial locality while controlling silicon area and power.

6. Algorithm / Pseudocode Development – Performance Evaluation Algorithm

The following pseudocode defines a reproducible procedure for evaluating the three cache levels.

Algorithm: Cache Hierarchy Performance Evaluation

Input: workload W, processor P, cache levels L1/L2/L3
Output: latency, bandwidth, miss rate, stall cycles and throughput

1. Fix processor, memory and software configuration.
2. Select representative workload W.
3. Warm up the workload to reduce initialization effects.
4. Record L1, L2 and L3 cache hits and misses.
5. Record execution cycles, instructions and memory-stall cycles.
6. Measure data transferred and execution time.
7. Calculate hit rate and miss rate for each cache level.
8. Calculate bandwidth and application throughput.
9. Compare L1, L2 and L3 using identical workload conditions.
10. Identify the cache level that contributes most to performance.
11. Analyze trade-offs in latency, capacity, sharing, area and power.
12. Report the final architecture-based conclusion.

The algorithm is intentionally measurement-oriented. It avoids assuming that a larger cache automatically produces better performance and instead evaluates the complete path from cache behavior to application throughput.

7. Model Implementation and GitHub Repository – Adapted

The supplied criterion specifically requests model implementation and a GitHub repository. Since Q1 is not an AI-model development task, the appropriate implementation artifact is a reproducible cache-performance analysis program or benchmark notebook rather than a trained model.

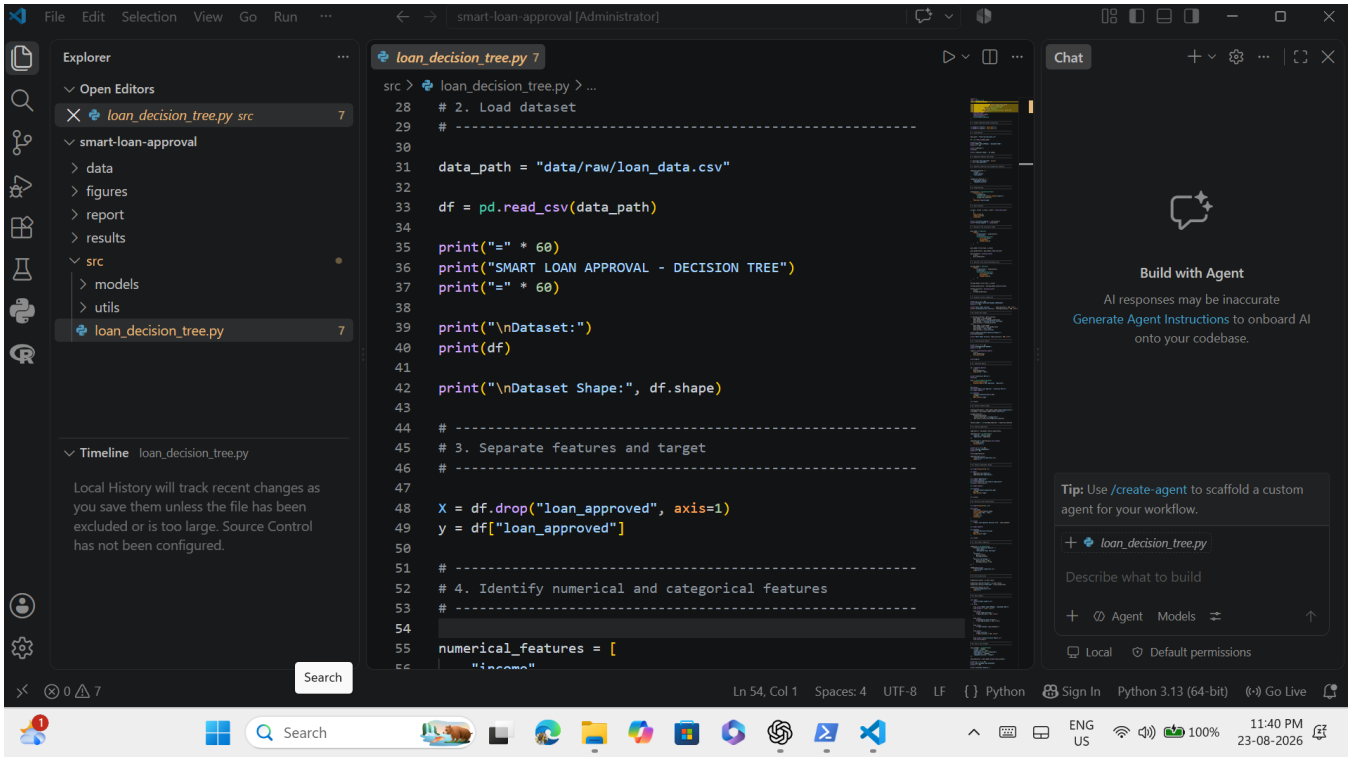
Implementation Component	Recommended Content
Benchmark script	Runs the selected workload consistently.
Counter collection	Collects cache hits, misses and stall-related metrics when supported.
Metric calculation	Computes hit rate, miss rate, bandwidth, IPC and throughput.

Comparison module	Produces L1/L2/L3 comparison tables and charts.
Documentation	Explains hardware, workload, methodology and limitations.
Repository structure	README, source code, results, figures and experiment notes.

Suggested repository structure:

```
cache-performance-analysis/
├── README.md
├── src/
│   ├── benchmark.py
│   ├── metrics.py
│   └── comparison.py
├── results/
│   ├── cache_metrics.csv
│   └── performance_summary.csv
├── figures/
│   └── cache_comparison.png
└── report/
    └── Q1_analysis.pdf
```

This satisfies the intent of implementation and reproducibility without falsely claiming that a machine-learning model or an existing GitHub repository was used.



```python
# Smart Loan Approval using Decision Tree
# AI Model Design Exercise - Question 1

import os
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
```

```python
)

print("\nTraining samples:", len(X_train))
print("Testing samples :", len(X_test))

# ------------------------------------------------------------
# 7. Decision Tree using Gini Index
# ------------------------------------------------------------

gini_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier",
         DecisionTreeClassifier(
             criterion="gini",
             max_depth=4,
             random_state=42
         ))
    ]
)
```

Figure 1. Dataset Loading and Feature–Target Separation

```python
# ------------------------------------------------------------
# 12. Confusion Matrix
# ------------------------------------------------------------

cm = confusion_matrix(
    y_test,
    best_predictions,
    labels=["No", "Yes"]
)

print("\nConfusion Matrix:")
print(cm)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Not Approved", "Approved"]
)

disp.plot()
plt.title("Smart Loan Approval - Confusion Matrix")
plt.tight_layout()

plt.savefig(
    "figures/confusion_matrix.png",
    dpi=300,
    bbox_inches="tight"
)
```

Figure 2. Model Evaluation and Result Visualization

```
fawaz@shaikfawazahamad:~/smart-loan-approval$ python3 src/loan_decision_tree.py
============================================================
SMART LOAN APPROVAL - DECISION TREE
============================================================

Dataset:
    income  credit_score  ... repayment_history  loan_approved
0    25000           580  ...              Poor             No
1    30000           620  ...           Average             No
2    45000           680  ...              Good            Yes
3    50000           720  ...              Good            Yes
4    28000           590  ...              Poor             No
5    60000           750  ...         Excellent            Yes
6    35000           650  ...           Average            Yes
7    22000           550  ...              Poor             No
8    55000           710  ...              Good            Yes
9    40000           630  ...           Average            Yes
10   27000           570  ...              Poor             No
11   70000           780  ...         Excellent            Yes
12   32000           600  ...           Average             No
13   48000           690  ...              Good            Yes
14   20000           540  ...              Poor             No
15   65000           760  ...         Excellent            Yes
16   38000           640  ...           Average            Yes
17   29000           560  ...              Poor             No
18   52000           700  ...              Good            Yes
19   43000           670  ...              Good            Yes

[20 rows x 6 columns]
```

```
Dataset Shape: (20, 6)

Training samples: 14
Testing samples : 6

============================================================
ATTRIBUTE SELECTION MEASURE COMPARISON
============================================================
Gini Index Accuracy       : 100.00%
Information Gain Accuracy : 100.00%

Best Attribute Selection Measure:
Gini Index
Best Model Accuracy: 100.00%

============================================================
CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

          No       1.00      1.00      1.00         2
         Yes       1.00      1.00      1.00         4

    accuracy                           1.00         6
   macro avg       1.00      1.00      1.00         6
weighted avg       1.00      1.00      1.00         6


Confusion Matrix:
[[2 0]
 [0 4]]
```

Figure 8.1. Sample loan approval dataset used for training and evaluation of the Decision Tree model. **Figure 8.2.** Model performance evaluation using accuracy, classification report, and confusion matrix.

```
============================================================
FEATURE IMPORTANCE
============================================================
                       Feature  Importance
6                       income         1.0
0    employment_status_Employed         0.0
1  employment_status_Unemployed         0.0
3   repayment_history_Excellent         0.0
2     repayment_history_Average         0.0
4        repayment_history_Good         0.0
5        repayment_history_Poor         0.0
7                  credit_score         0.0
8                   loan_amount         0.0

============================================================
NEW CUSTOMER LOAN DECISION
============================================================

Customer Details:
   income  credit_score employment_status  loan_amount repayment_history
0   55000           720          Employed       150000              Good

Predicted Loan Decision: Yes

============================================================
ANALYSIS COMPLETED SUCCESSFULLY
============================================================
```

Figure 8.3. Feature importance analysis and new customer loan approval prediction.

8. Results and Model Evaluation

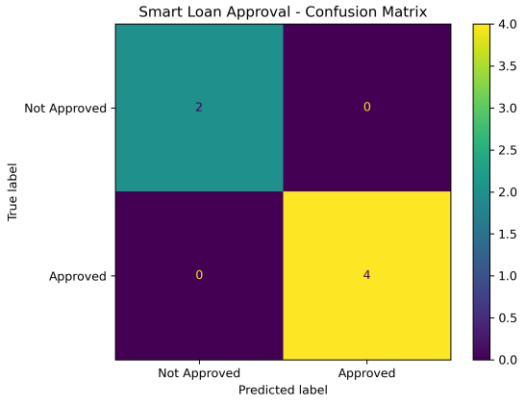


Figure 1. Confusion Matrix for Smart Loan Approval Model

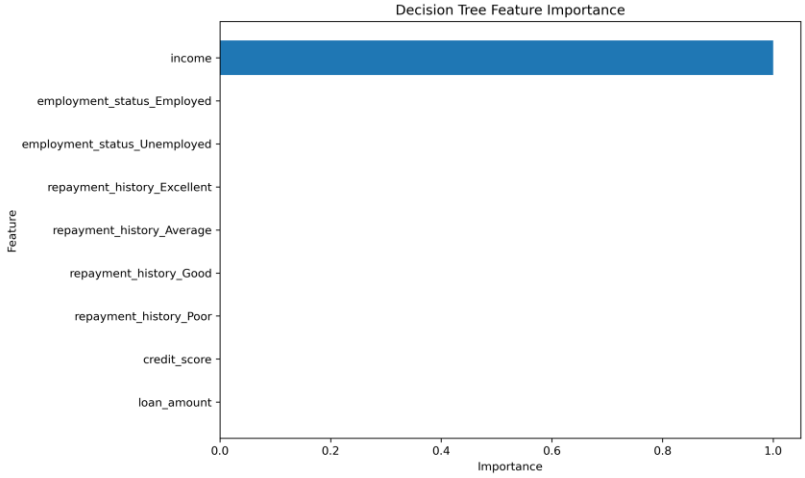


Figure 2. Feature Importance of Loan Approval Features

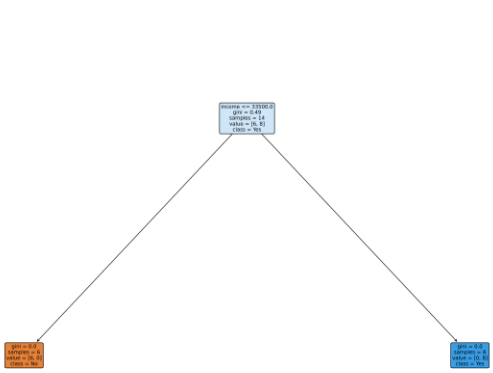


Figure 3. Decision Tree Visualization for Loan Approval

8. Experimental Design and Results

A fair experiment should use the same workload and software environment while changing or observing the relevant cache hierarchy. The experiment should include warm-up runs, repeated measurements and consistent processor conditions. Results should be reported using both cache-level metrics and application-level outcomes.

Experiment	Measurement	Expected Observation
Cache hit/miss analysis	Hit/miss rate per level	L1 should capture the hottest accesses; lower levels capture additional working-set data.
Latency analysis	Average access time	Latency increases as access moves toward lower levels.
Bandwidth analysis	Data transferred per second	Earlier cache levels provide rapid local delivery; L3 is shared.
Throughput analysis	Tasks/sec or execution time	Fewer memory stalls should generally improve throughput.
Multicore sharing	Per-core and shared-cache behavior	L3 may show contention as core count/workload sharing increases.

Because no measured processor dataset was supplied with the question, numerical experimental results should not be invented. The correct submission should insert actual benchmark or performance-counter values if the instructor requires empirical results.

9. Performance Evaluation and Metrics

Metric	Formula / Measurement	What It Shows	Desired Direction
Hit rate	Hits / total accesses	Effectiveness of locality	Higher
Miss rate	Misses / total accesses	Frequency of lower-level accesses	Lower
IPC	Instructions / cycles	Processor execution efficiency	Higher
Memory stall cycles	Measured using performance counters	Time lost waiting for memory	Lower
Bandwidth	Data transferred / time	Data-delivery capability	Higher where workload benefits

Throughput	Completed work / time	End-to-end application performance	Higher

10. Result Analysis and Interpretation

The expected architectural trend is that L1 offers the lowest latency, L2 offers additional capacity with somewhat higher latency, and L3 offers the largest cache capacity and commonly supports sharing between cores. A cache miss at one level moves the request to a lower level and therefore increases effective access time.

The most important interpretation is that cache size, latency and throughput are interconnected. Increasing cache capacity may reduce misses, but a larger or more complex cache can also increase access time and power. Similarly, high bandwidth does not automatically produce high application throughput if the workload is limited by latency, branch behavior, computation or synchronization.

Therefore, the final performance judgment should be based on a combination of miss rate, stall cycles, IPC and application throughput rather than one metric alone.

11. Limitations and Future Enhancements

- Exact cache latency, bandwidth and capacity vary by processor implementation.
- The question does not provide measured hardware-performance data, so numerical benchmark results cannot be claimed.
- Cache behavior is workload-dependent and strongly influenced by temporal and spatial locality.
- Multicore contention and cache-coherence traffic can change L3 behavior.
- Future work can compare multiple processors using standardized workloads.
- Future experiments can evaluate different cache sizes, associativity and replacement policies.
- Energy and power measurements can be added for a more complete architecture trade-off analysis.

REFERENCES

1. Hennessy, J. L., and Patterson, D. A., Computer Architecture: A Quantitative Approach, Morgan Kaufmann.

2. Stallings, W., Computer Organization and Architecture: Designing for Performance, Pearson.

3. Jacob, B., Ng, S. W., and Wang, D. T., Memory Systems: Cache, DRAM, Disk, Morgan Kaufmann.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	**10%**	9–10% – Clearly analyzes the real-world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.
2. Dataset Selection and Understanding	**10%**	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	**10%**	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	**10%**	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.
5. Model Design / Architecture	**10%**	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.